Visual Requirements Specification and Automated Test Generation for Digital Applications

Kapil Singi
Accenture Technology Labs
Accenture, India
Email: kapil.singi@accenture.com

Vikrant Kaulgud
Accenture Technology Labs
Accenture, India
Email: vikrant.kaulgud@accenture.com

Dipin Era
Accenture Technology Labs
Accenture, India
Email: dipin.era@accenture.com

***Abstract*—Digital, an amalgamation of social, cloud, mobility and analytics, is being widely used by enterprises for building innovative business applications. In digital applications, there is a healthy tension between 'business' and 'IT', leading to a rapidly evolving digital application. Often, digital applications have many stakeholders, requiring a role-based access and visualization of data. There is an emphasis on user experience. Due to such characteristics, a visual approach, for documenting and validating requirements is required. While this creates a better connect between business users and business analysts, it makes testing more difficult. In this paper, we present a rationale for visual requirements of digital applications, and then present a graph-model based approach for automated generation of test cases from visual requirements.**

I. Introduction

Gartner [1] says, 'Digital business disrupts IT, redefines the power of the consumer, and redraws the lines of competition across multiple industries. Businesses must act rapidly to respond to fundamental changes in competitive challenges, customer demands and technology use models'.

Digital applications are significantly different. First, due to the need for business to respond rapidly, the applications quickly evolve as per customer feedback. Second, since the business itself may not be fully aware of how to harness the synergistic capabilities of Digital, it is not in a position to specify complete requirements. Third, due to the emphasis on user experience, requirements typically have 'user delight' as a key concern.

In this context, traditional approaches of textual requirements [2] or use cases [3] are not entirely suitable. They capture what a system should do, but doesn't capture how a system should 'precisely' look and how it should behave for user stimuli and environmental stimuli (for instance, location context). In our experience, capturing the details relevant to Digital applications, requires a visual form of requirements specification.

In many cases, as much as 60% of code may consist of user interface code [4], [5], [6]. Although we do not have quantification, the big change we see in the IT industry is that a **very large portion of requirements specification itself** is now devoted to user experience. Tools such as Axure-RP [7] and JustInMind [8] provide intuitive support to business analysts for documenting requirements in a *visual format* (henceforth called prototype). The tools export prototypes to HTML5 applications which can be used on mobile devices or desktops *as if they were real applications*. Van den Bergh et al. [9] also commented on the advantages of interactive prototypes.

The other impetus to increasing use of such tools is a focus on '**Pretotyping**' [10]. Pretotyping largely depends on prototypes to help business stakeholders quickly rapidly converge on the right set of features and user experience.

While prototypes and pretotyping optimize the requirements phase of a Digital application development project, what is its impact on testing? Can test engineers, proficient in preparing test cases from textual requirements, be as effective in creating test cases from prototypes? What is the impact of prototypes on automated test generation tools?

These are key questions that emerge due to an increasing focus on Digital applications. In this paper, we present an approach to automatically generate test cases from prototypes and help test engineers adapt to Digital application testing. In Section 2, we highlight key challenges faced by test engineers in creating test cases from prototypes. In Section 3, we present APTGEN (Automated Prototype to Test-case GENerator), our approach to improving test engineers' productivity while testing Digital applications. We conclude with future work in Section 4.

II. Testing Visual Requirements Specifications

Prototypes captured using [7], [8] has two distinct perspectives of an application - 1) User Interface, and 2) Application Behaviour. The user interface (how the application looks) is specified using User Interface Components (UIC) such as text boxes, drop-down lists, radio buttons etc which are provided as palette within the prototype capturing tools. Digital applications use newer components such as location maps, data visualization etc. Through a set of screens consisting of a layout such UIC, one could completely specify the user interface. A very simple example is shown in Figure 1.

Application behaviour (how the application behaves) captures the effect of user stimuli, environmental stimuli, business rules etc. Prototyping tools like JustInMind provide two mechanisms for this. Basic behaviour such as 'go to Screen X if user clicks Button A' is configured visually on the tool. For advanced behaviour such as business rules, the tools provide a constrained natural language which allows business analysts to

978-1-4673-7073-8/15 $31.00 © 2015 IEEE
DOI 10.1109/RET.2015.15

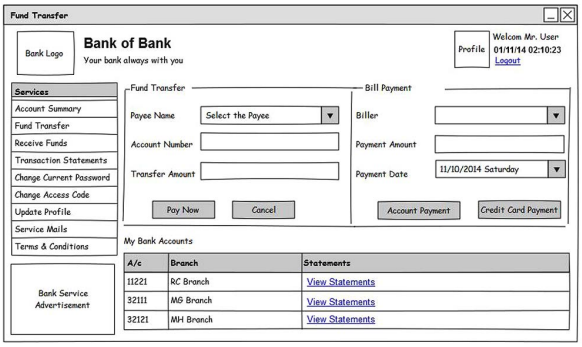

Fig. 1. A simple example of User Interface

describe rich conditions and actions. For instance, 'Pay Now' button in the interface in Figure 1, conditional event (type of application behaviour) could be coded as shown in Figure 2:

```
IF (PayeeName == "ABC" AND
    AccountNumber == "12345678"
    AND TransferAmount > 0)
THEN
     NAVIGATE TO Trx_Success_Screen
ELSE
     NAVIGATE TO Trx_Failure_Screen
```

Fig. 2. Conditional Event Application Behaviour

The important point to note is that application behaviour not only specifies rules, but can also embed data. When the prototypes are deployed as HTML5 applications, users can use this embedded data to interact with the application and precisely understand application behaviour.

A. Challenges in Testing

Given this background of prototypes, we discuss the challenges test engineers face in using protoypes for test case generation.

1) **Lack of documents to read**! Test engineers are accustomed to textual requirements for creating a mental model of the system. In prototype-based requirements, they are deprived of this key information source. The gap between the input to testing process (i.e. prototypes) and test engineer's skills has a **negative impact of productivity and test effectiveness**.
2) **Behavioural aspects are not explicitly visible**. Unless a test engineer opens a prototype in a native tool like JustInMind and is conversant with the constrained natural language, important rules would elude the test engineer. Without access to these rules, **test cases would be highly deficient**.
3) **Impact on test-case generation tools**. Since testing activities contribute to a large percentage (40-70%) of overall project cost [11]; automated test case generation helps improve productivity. Modeling techniques have been categorized by Manar et al. [12] as UML based, graph based, and specification and description based models (see [13] for a survey). Also, Nicha et al. [14] reviewed such techniques. Moreira et al. [15] proposed a GUI testing technique based on the UI patterns. However, **all these techniques works either on UML models, graph or description models as input and does not process specifications directly** for automated generation of test cases.

This motivates development of a tool for processing prototypes created by tools like JustInMind and automated generation of test cases, that test engineers could use for functional testing of Digital applications.

III. APTGEN

We now present Automated Prototype to Test-case GENerator (APTGEN), a tooling approach to tackle the challenges presented above, and improve the productivity of test engineers while testing Digital applications.

At a high-level, APTGEN takes prototypes and converts them into a set of graph models. These graph models are then analyzed along with semantic information pertaining to UIC and application behaviour rules to generate test cases. A template engine is used to generate human-readable test case descriptions. Figure 3 shows the complete APTGEN tool architecture.

The key processing steps performed in the APTGEN tool:

1) **Abstract model generation.** A prototype is a collection of interactive wire-frame screens. Each screen contains various UICs with the associated properties such as structural positioning on the screen, label, name, component-type etc. and the application behaviour rules such as conditional events. We **parse** the native format of a prototype file and extract features relevant to testing from the prototype. These features are stored in a canonical model; from which rest of the processing happens. The abstract screen model allows us to take inputs from a variety of prototype tools and reuse the core processing components of APTGEN. Following features are extracted:
   - Screen components such as application screens, pop-up windows etc.
   - Display components such as labels, images etc.
   - Input/Output components such as text-boxes, check-boxes, radio-buttons etc.
   - Action components such as buttons, links, images with overlaid hyperlinks etc.
   - Application events such as Navigate_to_screen, Call_Web_Service etc.

   Figure 4 depicts the UICs extracted from the sample screen shown earlier.
2) **Screen navigation model generation.** The navigation features link the screens into a navigation flow. For instance, on clicking a 'purchase' button, the navigation flows to a 'shopping cart' screen. The screen navigation

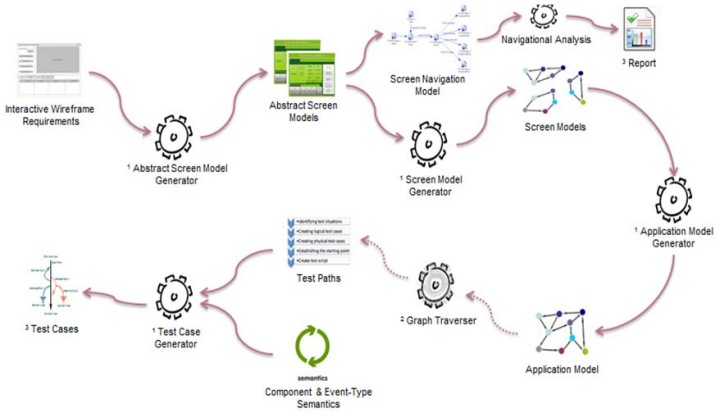

Fig. 3. APTGEN Architecture

ID	Type	Value
1	Screen	Fund Transfer
2	Screen	Transaction Success
3	Screen	Transaction Failed
4	Button	Pay Now
5	Button	Cancel
6	Button	Account Payment
7	Button	Credit Card Payment
8	ComboBox	Payee Name
9	TextBox	Account Number
10	TextBox	Transfer Amount
11	ComboBox	Biller
12	TextBox	Payment Amount
13	ComboBox	Payment Date
14	Label	Account Number
15	Label	Transfer Amount
234	Button	Watch Now
235	Image	Bank Logo
236	Image	Profile Logo

Fig. 4. Abstract Screen Model and Relationships

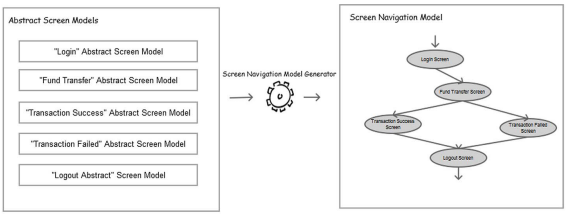

Fig. 5. Screen Navigation Model

model is depicted as a directed graph, where the nodes represent a screen, and a directed edge represents a path from one screen to another. While not specifically needed for test case generation, this model is useful for identifying orphan or unreachable screens. Also, one can find screens with high number of incoming edges. This signifies that such screens are critical to the application. The test engineers can use this information to plan adequate test coverage and effort. Figure 5 shows a sample screen navigation model generated for a e-commerce mobile app prototype containing multiple screens.

3) **Screen model generation.** For each screen in a prototype, such as that shown in Figure 1, we generate a graph model for all UICs on the screen. The graph is represented as a control flow graph, where each node is a UIC, and the edges represent relationships between UICs on the screen. Taking an example of the screen in Figure 1 and the behaviour in Figure 2, we would generate a screen model similar to that shown in Figure 6. The graph means that based on the positioning of the UICs, the user is most likely to enter payee name, enter account number, enter transfer amount. After that, the user can take one of the two actions - Pay Now or Cancel. If the user selects Pay Now, based on the application behaviour rule, the control navigates to *Trx_Success* screen or *Trx_Failure* screen.

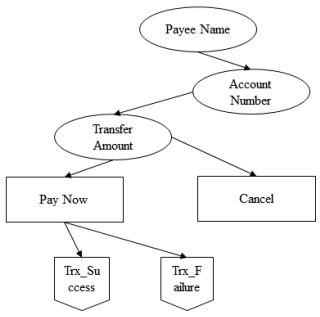

Fig. 6. A Partial Screen Model

The relationships between UIC for the sample screen in Figure 1 are shown using arrows on Figure 4. These relationships are used to generate the screen model. The visualized graph model shown in Figure 6 is a subset of a complex model that would be generated in reality.

4) **Application model generation.** Once all screen models

are generated, an application model is generated by combining the screen models. Steps used to generate the application model are:

a) Start processing from the initial Screen Model of the prototype. Use the state of the art graph traversal techniques, breadth first or depth first to traverse the Screen Model.
b) While traversing the Screen Model, on encountering the screen node for the different screen, For instance, Trx_Success or Trx_Failure, replace that screen node with its corresponding Screen Model generated in step 3.
c) Continue traversing the screen model graph, until all nodes are traversed. Perform step (b) on encountering of a screen node.
d) Once all the nodes are traversed, the final graph generated is the Application Model.

Application Model is a super-model consisting of all information pertaining to the prototype, including its constituent screens, UICs and UIC relations within a screen, application behavior etc. Figure 7 depicts generation of an Application Model from Screen Models.

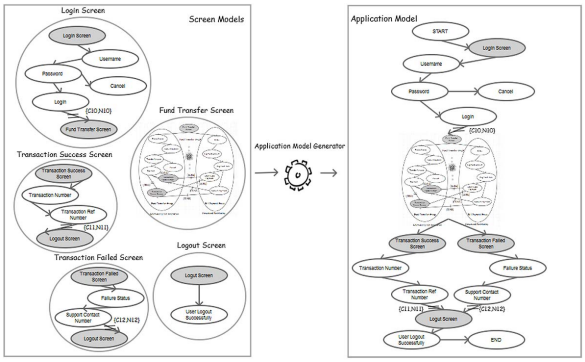

Fig. 7. Application Model

5) **Test path identification.** Once the application model is generated, we use state-of-art techniques such as that described by Dwarakanath and Jankiti [16], to identify optimal test paths.
6) **Test case generation.** However, since a test path is typically represented as a sequence of nodes and edges, by itself, a test path is not a suitable output for a test engineer. Here we leverage the 'semantics' of the nodes and edges to create a 'verbose' English language test-case description. For instance, for a text-box (an Input UIC), the semantic is of 'entering value'. Hence, for a text-box UIC node in a test path, we replace it with a template 'Enter #value# in the $text-box$ field'. Here #value# denotes that test-data will fill-in the template, and $text-box$ represents the particular text-box under consideration. Similarly, consider an application behaviour event like 'Hide'. The 'Hide' event has a semantic of hiding an UIC. Hence the related template looks like 'ASSERT that $Component$ is not visible.' Since 'Hide' is an action, we use ASSERT to verify that the action is indeed performed.

Finally, the entire output consisting of multiple test-cases for a prototype is populated in an Excel spreadsheet. The test engineer can verify the test cases, and augment the test cases using tacit knowledge. Later the test-cases could be stored in a test management tool like HP Quality Center.

IV. Conclusion and Future Work

In this paper, we first articulate the change Digital brings to how requirements are captured and documented. We see that the need to be agile puts significant emphasis on using visual requirements specification (prototypes). However, this has an impact on testing of Digital applications. To improve productivity in testing, it is essential to automatically generate test cases from prototypes. We presented **APTGEN** for this task. We are currently piloting APTGEN in real-life projects. Based on the results, we will further refine our techniques. We are also developing a set of annotations that would help test engineers augment the prototypes with more business conditions, functional knowledge etc. and make test cases even more relevant. Using APTGEN, it is possible to complement the 'pretotyping' concept and achieve truly agile requirements and testing phases.

References

[1] Gartner, "Gartner predicts 2015," 2015, http://www.gartner.com/technology/research/predicts.
[2] V. Ambriola and V. Gervasi, "Processing natural language requirements," in *Automated Software Engineering, 1997. Proceedings., 12th IEEE International Conference*. IEEE, 1997, pp. 36–45.
[3] I. Jacobson, "Object oriented software engineering: a use case driven approach," 1992.
[4] R. Mahajan and B. Shneiderman, "Visual and textual consistency checking tools for graphical user interfaces," *Software Engineering, IEEE Transactions on*, vol. 23, no. 11, pp. 722–735, 1997.
[5] B. A. Myers, "User interface software tools," *ACM Transactions on Computer-Human Interaction (TOCHI)*, vol. 2, no. 1, pp. 64–103, 1995.
[6] B. Myers, *State of the art in user interface software tools*. Citeseer, 1992.
[7] R. Axure, "Interactive wireframe software and mockup tool," 2012.
[8] "Justinmind." [Online]. Available: http://www.justinmind.com/
[9] J. Van den Bergh, D. Sahni, M. Haesen, K. Luyten, and K. Coninx, "Grip: get better results from interactive prototypes," in *Proceedings of the 3rd ACM SIGCHI symposium on Engineering interactive computing systems*. ACM, 2011, pp. 143–148.
[10] S. Alberto, "Pretotype it," http://goo.gl/HA65GP, 2011.
[11] L. Mich, M. Franch, and P. Novi Inverardi, "Market research for requirements analysis using linguistic tools," *Requirements Engineering*, vol. 9, no. 2, pp. 151–151, 2004.
[12] M. H. Alalfi, J. R. Cordy, and T. R. Dean, "Modelling methods for web application verification and testing: state of the art," *Software Testing, Verification and Reliability*, vol. 19, no. 4, pp. 265–296, 2009.
[13] M. Prasanna, S. Sivanandam, R. Venkatesan, and R. Sundarrajan, "A survey on automatic test case generation," *Academic Open Internet Journal*, vol. 15, no. part 6, 2005.
[14] N. Kosindrdecha and J. Daengdej, "A test case generation process and technique," *J. Software Eng*, vol. 4, pp. 265–287, 2010.
[15] R. M. Moreira, A. C. Paiva, and A. Memon, "A pattern-based approach for gui modeling and testing," in *Software Reliability Engineering (ISSRE), 2013 IEEE 24th International Symposium on*. IEEE, 2013, pp. 288–297.
[16] A. Dwarakanath and A. Jankiti, "Minimum number of test paths for prime path and other structural coverage criteria," in *Testing Software and Systems*. Springer, 2014, pp. 63–79.